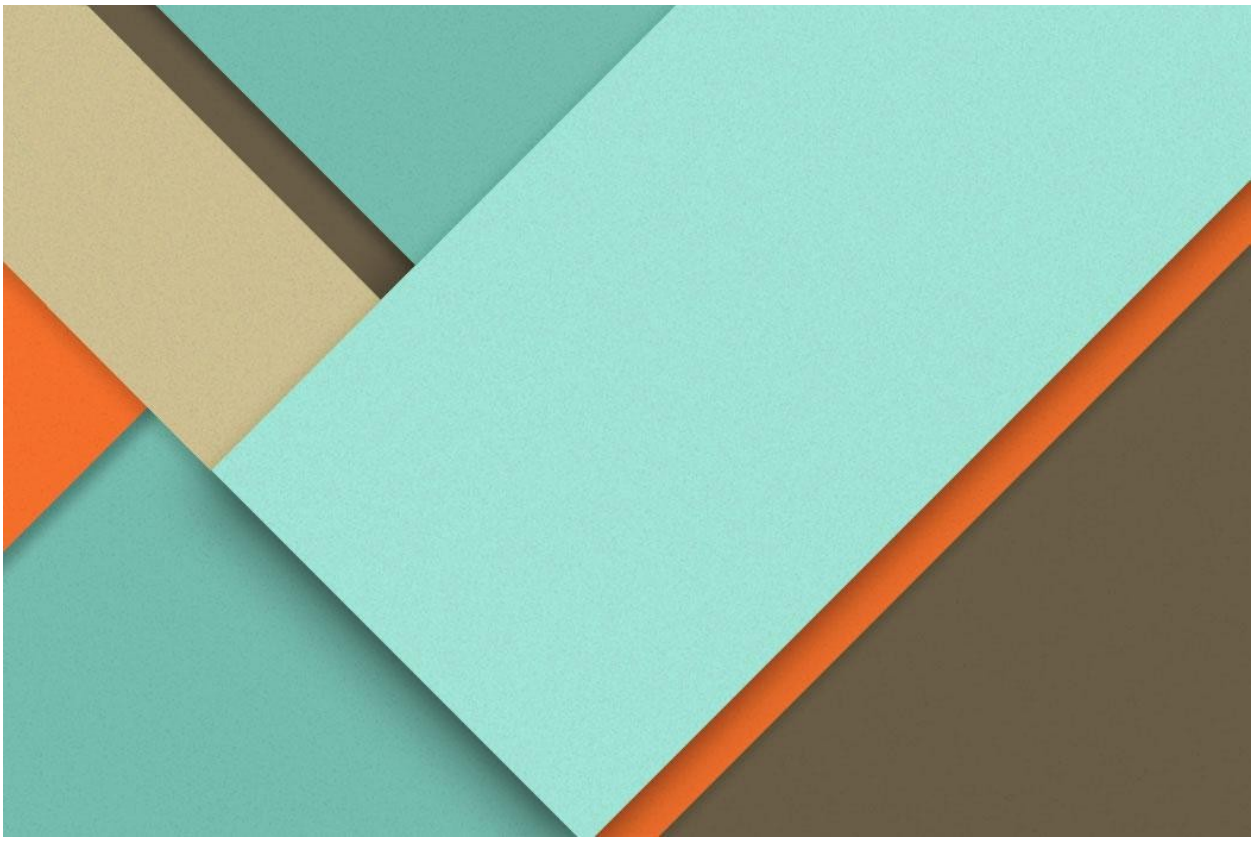


Self-Healing Infrastructure with Auto Scaling using AWS Cloud

Submitted by:
DANDUGULA CHANDU
Roll Number: 245123749038
245123749038@mvsrec.edu.in

Under the Guidance of:
[Guide Name]

Department:
CSE (IoT)

College Name
MVSR Engineering College, Hyderabad
Osmania University, Hyderabad
Academic Year: 2026-27

2. Certificate

This is to certify that the project report titled **"Self-Healing Infrastructure with Auto Scaling using AWS Cloud"** submitted by **DANDUGULA CHANDU** (Roll No. **245123749038**) in partial fulfillment of the requirements for the award of the **Bachelor of Technology** degree has been carried out under my supervision.

Guide Signature	HOD Signature	Principal Signature

3. Acknowledgment

I would like to express my sincere gratitude to my project guide [Guide Name] for their continuous support, guidance, and encouragement throughout the development of this project.

I also thank the Department of CSE (IoT) and **MVSR Engineering College** for providing the necessary facilities to complete this work successfully.

Finally, I thank my friends and family members for their support during the project.

DANDUGULA CHANDU

4. Abstract

Cloud computing has revolutionized how applications and infrastructure are deployed and managed. Traditional IT infrastructures depend on manual monitoring to detect failures and to handle changing workloads, which leads to delayed recovery, unplanned downtime, and servers that are either over-provisioned or overloaded.

This project presents **Self-Healing Infrastructure with Auto Scaling using AWS Cloud**, which automatically detects system failures and dynamically adjusts computing resources based on demand. It uses **Amazon EC2** for compute, **Elastic Load Balancing** for traffic distribution and health checks, an **Auto Scaling Group** for capacity management, **Amazon CloudWatch** for metrics and alarms, **Amazon SNS** for notifications, and a **Virtual Private Cloud** for network isolation.

The infrastructure continuously monitors the health of its servers and automatically replaces failed instances without human intervention. It also adds instances during traffic spikes and removes them when demand falls, improving availability, performance and cost efficiency.

The project delivers deployable code (a CloudFormation template, EC2 start-up script, and failure/load test scripts) and a prototype simulation of the same design. In the prototype, a failed instance was detected in **15 s** and fully replaced in **105 s**, and under a traffic spike the auto-scaled system served **98.9%** of requests compared with **74.2%** for a static two-server deployment, while using **20.7% fewer instance-minutes** than a statically over-provisioned five-server deployment.

5. Table of Contents

6. Introduction
7. Literature Survey
8. Problem Statement
9. Objectives
10. Existing System
11. Proposed System
12. System Architecture
13. Technologies Used
14. Implementation Methodology
15. Source Code
16. System Testing
17. Results and Discussion
18. Advantages and Limitations
19. Applications
20. Future Scope
21. Conclusion
22. References

Appendix A - Complete CloudFormation Template

Appendix B - Prototype Simulation Source Code

6. Introduction

6.1 Background

Cloud computing has become one of the most important technologies in modern computing environments. Organizations rely heavily on cloud infrastructure to host applications, store data, and provide services to users worldwide. Applications such as e-commerce sites, learning portals and banking services are expected to be available around the clock, and even a few minutes of downtime can cause lost revenue and damage to reputation.

Traditional infrastructure requires manual system management: administrators watch dashboards, detect failed servers and launch replacements by hand. Traffic is also unpredictable. A sale, an exam result or a viral post can multiply load within minutes, while at night the same servers sit mostly idle.

6.2 Need for the Project

Modern applications require:

- **High availability** - the service keeps running when a component fails
- **Automatic scaling** - capacity follows demand
- **Fault tolerance** - failures in one server or Availability Zone do not stop the service
- **Reduced operational cost** - pay only for the capacity actually needed

A **Self-Healing Infrastructure** ensures that system failures are detected automatically and replaced without human intervention, while **Auto Scaling** adds or removes servers to match the workload.

6.3 Cloud Computing Model

Three major service models exist: **IaaS** (Infrastructure as a Service), **PaaS** (Platform as a Service) and **SaaS** (Software as a Service). This project mainly uses **Infrastructure as a Service (IaaS)**: virtual servers (EC2), networking (VPC) and load balancing are provisioned and configured directly, giving full control over scaling and recovery behaviour.

---

7. Literature Survey

Scalable and fault-tolerant cloud infrastructure has been widely studied. Armbrust et al. identified elasticity - the ability to add and remove resources on demand without long-term commitment - as a defining property of cloud computing, and the NIST definition of cloud computing lists rapid elasticity and measured service among its essential characteristics.

Cloud providers turned these ideas into managed services. **Amazon EC2 Auto Scaling** maintains a group of instances between a minimum and maximum size, replaces instances that fail health checks, and adds or removes capacity using scaling policies. **Elastic Load Balancing** distributes traffic across healthy targets and reports target health, and **Amazon CloudWatch** collects metrics and raises alarms that can trigger scaling actions. The AWS Well-Architected Framework (Reliability pillar) recommends automatic recovery from failure and horizontal scaling as design principles.

Large-scale operators apply the same philosophy. Netflix, for example, published its Chaos Monkey tool, which deliberately terminates production instances so that engineers are forced to build services that tolerate instance failure. The failure test used in this project (Section 16) follows the same idea on a small scale.

Infrastructure as Code tools such as AWS CloudFormation allow a whole environment to be described in a template and recreated consistently, which is the approach used for the deployable part of this project.

---

8. Problem Statement

Traditional server infrastructures suffer from the following problems:

- **Manual system monitoring** - failures are noticed late
- **High downtime during failures** - recovery depends on an administrator being available
- **Limited scalability** - fixed capacity cannot absorb sudden spikes
- **High operational cost** - servers are sized for peak load and idle the rest of the time

Therefore, there is a need for an automated cloud infrastructure that can detect failures, repair itself, and adjust resources dynamically.

---

9. Objectives

- To design a scalable, highly available cloud infrastructure on AWS
- To implement automatic server failure recovery (self-healing)
- To implement auto scaling based on traffic demand using CloudWatch alarms
- To distribute traffic across multiple Availability Zones with a load balancer
- To reduce manual system maintenance and operating cost
- To provide repeatable deployment code and test scripts, and to evaluate behaviour through a prototype

---

10. Existing System

Traditional hosting environments rely on single servers or manually managed infrastructure. Capacity is fixed in advance, monitoring is done by people, and recovery means logging in to a failed server or building a new one.

Limitations

- No automatic failure recovery
- Difficult to handle traffic spikes
- High downtime risk because a single server is a single point of failure
- Manual monitoring required
- Capacity is wasted during quiet periods

11. Proposed System

The proposed system implements **Self-Healing Infrastructure using AWS services**. Key features include:

- **Automatic server replacement** - the Auto Scaling Group uses the load balancer's health check and replaces any instance that fails it
- **Dynamic scaling based on demand** - CloudWatch CPU alarms add one instance above 70% average CPU and remove one below 30%
- **Load balancing across multiple servers** - an Application Load Balancer sends traffic only to healthy instances
- **Multi-AZ deployment** - instances run in two Availability Zones
- **Continuous monitoring and alerts** - CloudWatch metrics with SNS e-mail notifications
- **Consistent configuration** - a launch template guarantees every replacement is identical

The system uses **AWS EC2 instances, Application Load Balancer, Auto Scaling Group, CloudWatch, SNS and a VPC** to achieve these capabilities.

12. System Architecture

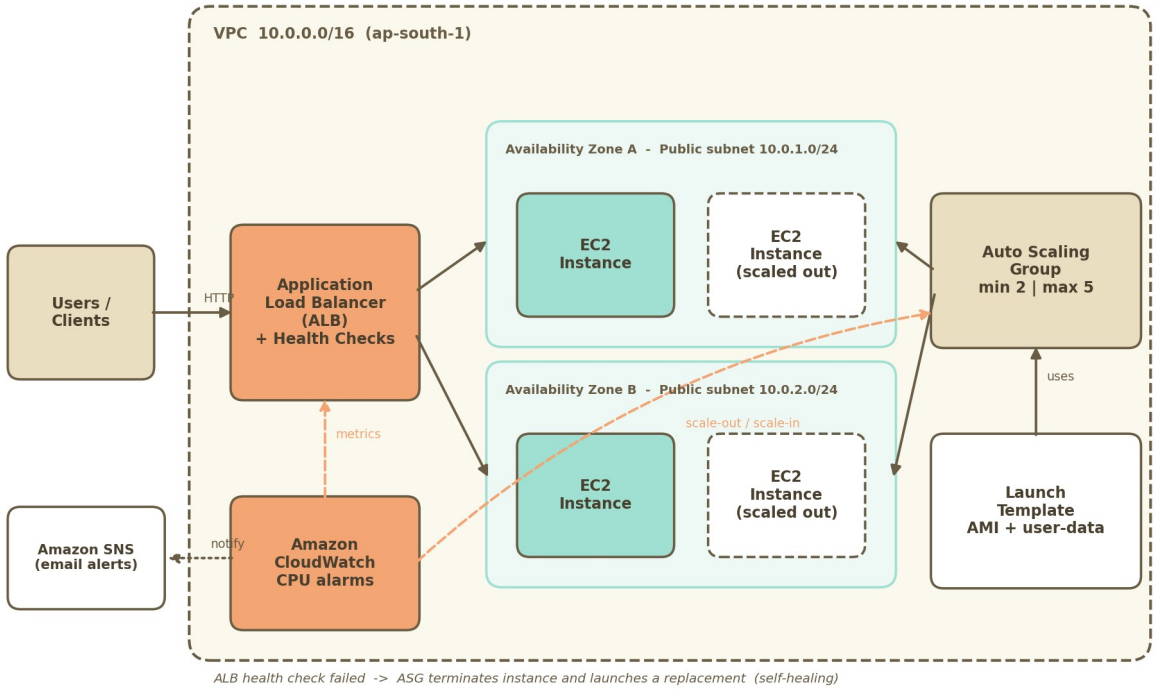


Figure 12.1 - Architecture of the Self-Healing Infrastructure with Auto Scaling

12.1 Components

Component	Role in the system
Users / Clients	Browsers or apps that generate the traffic the system must serve.
Application Load Balancer	Single entry point (port 80). Distributes requests across healthy instances and stops sending traffic to an instance that fails its health check.
Target Group + Health Check	Instances registered behind the ALB. Health check: HTTP GET /health.html every 15 s; 2 failures = unhealthy, 2 successes = healthy.
EC2 Instances	Amazon Linux 2 servers (t2.micro) running Apache, spread over two Availability Zones.
Launch Template	Defines AMI, instance type, security group and start-up script so every new instance is identical.
Auto Scaling Group	Keeps 2 to 5 instances running (desired 2). Health check type = ELB, so an instance failing the ALB check is terminated and replaced.
Amazon CloudWatch	Collects CPU metrics; two alarms drive the scale-out and scale-in policies.
Scaling Policies	Scale-out +1 instance (cooldown 120 s); scale-in -1 instance (cooldown 300 s).
VPC, Subnets, Security Groups	VPC 10.0.0.0/16 with subnets 10.0.1.0/24 (AZ a) and 10.0.2.0/24 (AZ b). Instances accept HTTP only from the ALB's security group.
Amazon SNS	E-mail notification on instance launch, termination and launch errors.

12.2 Data Flow

A user request reaches the ALB, which forwards it to a healthy instance in either Availability Zone. In parallel, the ALB probes each instance every 15 s. If an instance fails two consecutive probes, it is marked unhealthy and receives no traffic; the Auto Scaling Group then terminates it and launches a replacement from the launch template. Separately, CloudWatch evaluates the group's average CPU every minute and triggers the scale-out or scale-in policy when a threshold is crossed.

13. Technologies Used

Technology	Description
AWS EC2	Virtual servers that run the web application (t2.micro, Amazon Linux 2).
Elastic Load Balancer (ALB)	Layer-7 traffic distribution and health checking.
Auto Scaling Group	Dynamic scaling and automatic replacement of failed instances.
Amazon CloudWatch	Monitoring metrics and alarms.
Amazon SNS	E-mail notifications.
VPC	Secure networking: subnets, route table, internet gateway, security groups.
AWS CloudFormation	Infrastructure as Code - the whole stack is created from one YAML template.
Linux + Apache httpd	Operating system and web server on each instance.
Python 3 (boto3, matplotlib)	Failure-injection script, load-test script, and the prototype simulation with charts.
Bash / AWS CLI	Instance start-up script and deployment commands.

14. Implementation Methodology

The environment can be built manually from the AWS Management Console (Steps 1-8) or created in one command from the CloudFormation template (Step 9). Region used in the examples: **ap-south-1 (Mumbai)**.

Step 1: Create VPC

Create a Virtual Private Cloud with CIDR block `10.0.0.0/16`, enable DNS hostnames, and attach an Internet Gateway.

Step 2: Create Subnets

Create two subnets in different Availability Zones: `10.0.1.0/24` (AZ a) and `10.0.2.0/24` (AZ b). Add a route table with route `0.0.0.0/0` to the Internet Gateway and associate both subnets. The load balancer needs subnets in at least two AZs.

Design note: to avoid NAT Gateway cost in a student environment, the EC2 instances are placed in the same public subnets as the ALB, but their security group accepts HTTP **only from the ALB's security group**. A production design would place instances in private subnets behind a NAT Gateway.

Step 3: Create Security Groups

`alb-sg`: allow inbound TCP 80 from `0.0.0.0/0`. `instance-sg`: allow inbound TCP 80 only from `alb-sg`.

Step 4: Create Launch Template and Web Server

Create a launch template (Amazon Linux 2 AMI, t2.micro, `instance-sg`) with the following user data. It installs Apache, publishes a page that shows the instance ID and Availability Zone, and creates the `/health.html` health-check page. The core commands are:

Core commands

```
sudo yum update -y
sudo yum install httpd -y
sudo systemctl start httpd
sudo systemctl enable httpd
```

The complete script is listed in Section 15.1.

Step 5: Configure Target Group and Application Load Balancer

Create a target group (HTTP:80) with health check path `/health.html`, interval 15 s, healthy threshold 2, unhealthy threshold 2. Create an internet-facing Application Load Balancer in both subnets with a listener on port 80 forwarding to the target group.

Step 6: Create Auto Scaling Group

Create the group from the launch template: **minimum 2, desired 2, maximum 5**, both subnets, attached to the target group. Under health checks, **turn on ELB health checks** (otherwise only EC2 status checks are used and an instance whose web server has crashed would not be replaced) and set the health check grace period to 120 s.

Step 7: Configure Scaling Policies and CloudWatch Alarms

Alarm	Condition	Action
self-healing-cpu-high	Average CPU > 70% Period 60 s x 2	Scale out: add 1 instance, cooldown 120 s; notify SNS topic
self-healing-cpu-low	Average CPU < 30% Period 300 s x 2	Scale in: remove 1 instance, cooldown 300 s

Step 8: Configure SNS Notifications

Create an SNS topic, subscribe an e-mail address (and confirm the subscription e-mail), and add it as the notification target of the Auto Scaling Group for launch, terminate and launch-error events.

Step 9: One-command deployment (alternative)

Deploy with CloudFormation

```
cd scripts
./deploy.sh up your-email@example.com   # creates the whole stack
# ... run the tests ...
./deploy.sh down                        # deletes everything
```

Cost note: the Application Load Balancer and EC2 instances may incur charges depending on your account's free-tier eligibility. Delete the stack after testing.

15. Source Code

All source files are supplied in the project folder. The structure is:

Project structure

```
self-healing-aws-project/
  cloudformation/self-healing-asg.yaml   (VPC, ALB, ASG, alarms, SNS - Appendix A)
  scripts/user-data.sh                   (EC2 start-up script)
  scripts/deploy.sh                      (create / delete the stack)
  scripts/test_instance_failure.py       (Test Case 1 - failure injection)
  scripts/load_test.py                   (Test Case 2 - traffic generator)
  scripts/cpu_stress_on_instance.sh      (optional CPU stress)
  simulation/simulate.py                 (prototype simulation - Appendix B)
  simulation/analyze.py, make_arch.py    (statistics, architecture diagram)
  simulation/output/                     (CSV data, event logs, charts)
```

15.1 EC2 start-up script (user-data.sh)

scripts/user-data.sh

```
#!/bin/bash
# EC2 user data: installs Apache and publishes a page showing which instance/AZ served the request
yum update -y
yum install -y httpd stress
systemctl enable --now httpd

TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/instance-id)
AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-data/placement/availability-zone)

cat > /var/www/html/index.html <<HTML
<html><body style="font-family:Arial;text-align:center;margin-top:60px">
<h1>Self-Healing Infrastructure with Auto Scaling</h1>
<h2>Served by instance: $IID</h2>
<h3>Availability Zone: $AZ</h3>
</body></html>
HTML

# Health check endpoint used by the ALB target group
echo "OK" > /var/www/html/health.html
```

15.2 Key part of the CloudFormation template

The Auto Scaling Group and CloudWatch alarm definitions are the heart of the design. Note `HealthCheckType: ELB`, which makes the group replace instances that fail the load balancer's health check. The complete template is in Appendix A.

cloudformation/self-healing-asg.yaml (excerpt)

```
  AutoScalingGroup:
    Type: AWS::AutoScaling::AutoScalingGroup
    Properties:
      AutoScalingGroupName: self-healing-asg
      MinSize: '2'
      MaxSize: '5'
```

```
    DesiredCapacity: '2'
    HealthCheckType: ELB
    HealthCheckGracePeriod: 120
    VPCZoneIdentifier: [!Ref PublicSubnetA, !Ref PublicSubnetB]
    TargetGroupARNs: [!Ref TargetGroup]
    LaunchTemplate:
      LaunchTemplateId: !Ref LaunchTemplate
      Version: !GetAtt LaunchTemplate.LatestVersionNumber
    NotificationConfigurations:
      - TopicARN: !Ref AlertTopic
        NotificationTypes:
          - autoscaling:EC2_INSTANCE_LAUNCH
          - autoscaling:EC2_INSTANCE_TERMINATE
          - autoscaling:EC2_INSTANCE_LAUNCH_ERROR

ScaleOutPolicy:
  Type: AWS::AutoScaling::ScalingPolicy
  Properties:
    AutoScalingGroupName: !Ref AutoScalingGroup
    PolicyType: SimpleScaling
    AdjustmentType: ChangeInCapacity
    ScalingAdjustment: 1
    Cooldown: '120'
ScaleInPolicy:
  Type: AWS::AutoScaling::ScalingPolicy
  Properties:
    AutoScalingGroupName: !Ref AutoScalingGroup
    PolicyType: SimpleScaling
    AdjustmentType: ChangeInCapacity
    ScalingAdjustment: -1
    Cooldown: '300'

HighCpuAlarm:
  Type: AWS::CloudWatch::Alarm
  Properties:
    AlarmName: self-healing-cpu-high
    Namespace: AWS/EC2
    MetricName: CPUUtilization
    Dimensions: [{Name: AutoScalingGroupName, Value: !Ref AutoScalingGroup}]
    Statistic: Average
    Period: 60
    EvaluationPeriods: 2
    Threshold: 70
    ComparisonOperator: GreaterThanThreshold
    AlarmActions: [!Ref ScaleOutPolicy, !Ref AlertTopic]
LowCpuAlarm:
  Type: AWS::CloudWatch::Alarm
  Properties:
    AlarmName: self-healing-cpu-low
    Namespace: AWS/EC2
    MetricName: CPUUtilization
    Dimensions: [{Name: AutoScalingGroupName, Value: !Ref AutoScalingGroup}]
    Statistic: Average
    Period: 300
    EvaluationPeriods: 2
    Threshold: 30
    ComparisonOperator: LessThanThreshold
    AlarmActions: [!Ref ScaleInPolicy]
```

15.3 Failure-injection script (Test Case 1)

scripts/test_instance_failure.py

```
"""Test Case 1 - terminate one instance and measure how long the ASG takes to heal.
Usage: python test_instance_failure.py [asg-name] [region]
"""
import sys, time, datetime
import boto3

asg_name = sys.argv[1] if len(sys.argv) > 1 else "self-healing-asg"
region = sys.argv[2] if len(sys.argv) > 2 else "ap-south-1"
asg = boto3.client("autoscaling", region_name=region)
ec2 = boto3.client("ec2", region_name=region)

def in_service():
    g = asg.describe_auto_scaling_groups(AutoScalingGroupNames=[asg_name])["AutoScalingGroups"][0]
    return g["DesiredCapacity"], [i["InstanceId"] for i in g["Instances"] if i["LifecycleState"] ==
"InService"]

def log(msg):
    print(f"[{datetime.datetime.now():%H:%M:%S}] {msg}", flush=True)

desired, ids = in_service()
log(f"Desired={desired}, InService={len(ids)} -> {ids}")
victim = ids[0]
log(f"Simulating failure: terminating {victim}")
ec2.terminate_instances(InstanceIds=[victim])
t0 = time.time()

while True:
    time.sleep(10)
    d, now = in_service()
    log(f"Desired={d}, InService={len(now)} -> {now}")
    if len(now) == desired and victim not in now:
        log(f"SELF-HEALED in {int(time.time() - t0)} seconds. Replacement: {set(now) - set(ids)}")
        break
```

15.4 Load-test script (Test Case 2)

scripts/load_test.py

```
"""Test Case 2 - generate HTTP load against the ALB so that CPU rises and the ASG scales out.
Usage: python load_test.py http://<alb-dns> [threads] [seconds]
"""
import sys, time, threading, collections, urllib.request

url = sys.argv[1]
threads = int(sys.argv[2]) if len(sys.argv) > 2 else 50
duration = int(sys.argv[3]) if len(sys.argv) > 3 else 300
ok = err = 0
served_by = collections.Counter()
lock = threading.Lock()
stop = time.time() + duration

def worker():
    global ok, err
```

```
    while time.time() < stop:
        try:
            body = urllib.request.urlopen(url, timeout=5).read().decode()
            with lock:
                ok += 1
                if "nstance" in body:
                    served_by[body.split("nstance")[1][:25]] += 1
        except Exception:
            with lock:
                err += 1

ts = [threading.Thread(target=worker) for _ in range(threads)]
[t.start() for t in ts]
while time.time() < stop:
    time.sleep(10)
    print(f"requests ok={ok} errors={err}", flush=True)
[t.join() for t in ts]
print(f"DONE: ok={ok} errors={err}")
for k, v in served_by.most_common():
    print(f"  {k.strip()}: {v}")
```

15.5 Deployment script

scripts/deploy.sh

```
#!/bin/bash
# Deploy / delete the stack. Requires AWS CLI configured (aws configure).
set -e
STACK=self-healing-demo
case "$1" in
  up)
    aws cloudformation deploy --stack-name $STACK \
      --template-file ../cloudformation/self-healing-asg.yaml \
      --parameter-overrides AlertEmail="$2"
    aws cloudformation describe-stacks --stack-name $STACK \
      --query "Stacks[0].Outputs" --output table ;;
  down)
    aws cloudformation delete-stack --stack-name $STACK
    aws cloudformation wait stack-delete-complete --stack-name $STACK
    echo "Stack deleted - no further charges." ;;
  *) echo "Usage: $0 up <alert-email> | down" ;;
esac
```

16. System Testing

Testing ensures the system works correctly. Five test cases were defined. Test Cases 1-3 were executed on the **prototype simulation** (Section 17); Test Cases 4-5 are designed for the live AWS deployment.

Test Case 1 - Instance Failure

Procedure: terminate one EC2 instance (`test_instance_failure.py`) while traffic is flowing. **Expected result:** the ALB stops routing to it, and Auto Scaling launches a new instance to restore the desired capacity.

Test Case 2 - High Traffic

Procedure: generate load (`load_test.py` or `stress`) until average CPU exceeds 70%. **Expected result:** the CloudWatch alarm fires and new instances are created, up to the maximum of 5.

Test Case 3 - Traffic Drop

Procedure: stop the load. **Expected result:** CPU stays below 30% and the group scales in toward the minimum, subject to the cooldown.

Test Case 4 - Application-level failure

Procedure: log in to one instance and run `sudo systemctl stop httpd` (the instance is running but the web server is down). **Expected result:** the ALB health check fails, the instance is marked unhealthy, and - because the group uses ELB health checks - it is replaced.

Test Case 5 - Availability Zone loss

Procedure: terminate all instances in one AZ. **Expected result:** the site stays up on the other AZ, and the group relaunches instances to restore capacity.

ID	Test	Pass criterion	Status
TC1	Instance failure	Replacement InService; site stays reachable	Simulated - passed
TC2	High traffic	Scale-out events; availability improves vs static	Simulated - passed
TC3	Traffic drop	Scale-in after sustained low CPU	Simulated - passed
TC4	Application-level failure	Instance replaced after failed health checks	Run on AWS: [pending]
TC5	AZ loss	Site stays up; capacity restored	Run on AWS: [pending]

17. Results and Discussion

This section reports the results of the **prototype simulation** (`simulation/simulate.py`). The simulation models the ALB, Auto Scaling Group and CloudWatch alarms using the same parameters as the CloudFormation template, and compares the design against static deployments. **These are results from a model, not measurements from a live AWS account**; the modelling assumptions are listed below.

17.1 Simulation assumptions

Parameter	Value
Capacity of one instance	200 requests/s at 100% CPU (assumed)
Instance boot time (launch + user-data)	75 s (assumed)
ALB health check	Every 15 s; 2 failures = unhealthy; 2 successes = healthy
Scale-out alarm	Average CPU > 70% for 2 x 60 s; +1 instance; cooldown 120 s
Scale-in alarm	Average CPU < 30% for 2 x 300 s; -1 instance; cooldown 300 s
Group size	Min 2, desired 2, max 5
Manual recovery of the static baseline	900 s (15 min) after failure (assumed)

17.2 Test 1 - Instance failure

Traffic is steady at about 220 requests/s across two instances. At minute 5, one instance crashes.

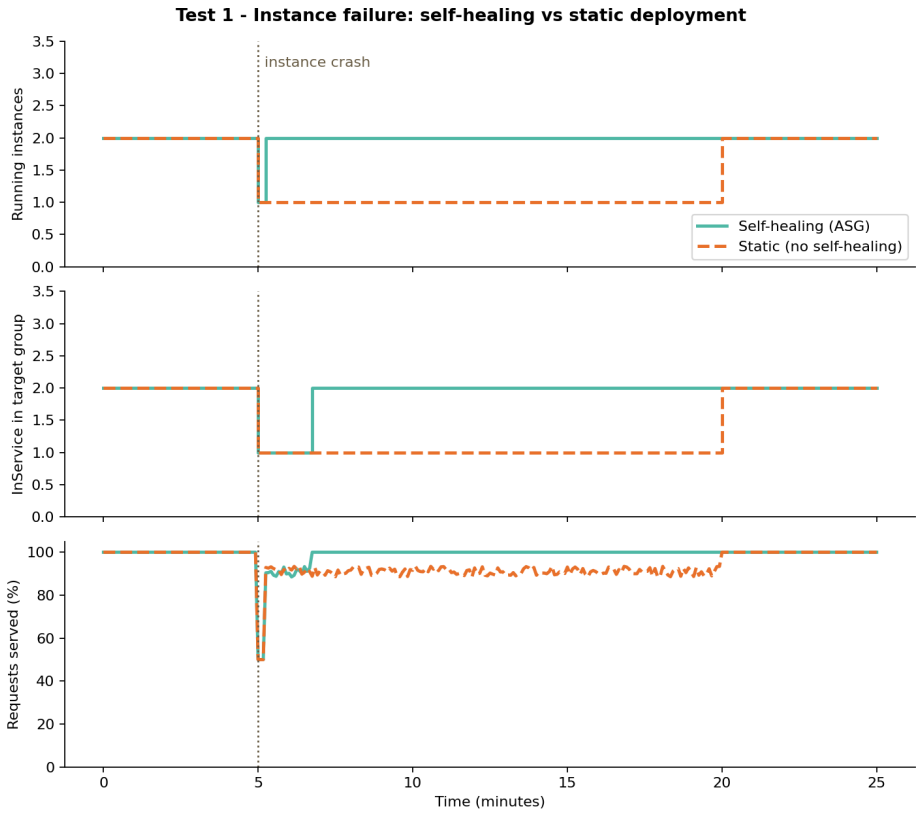


Figure 17.1 - Self-healing (solid) versus a static deployment with manual recovery (dashed)

Event log - self-healing system (simulation output)

```
[T+00:00] ASG started: desired=2, instances=['i-0a1b4b2c', 'i-0a1b6a1b']
[T+05:00] FAILURE injected: i-0a1b4b2c (ap-south-1a) crashed
[T+05:15] ALB: i-0a1b4b2c FAILED 2 health checks -> marked Unhealthy, traffic stopped
[T+05:15] ASG: terminating unhealthy i-0a1b4b2c
[T+05:15] ASG: launching replacement i-0a1b890a (ap-south-1a) from launch template
[T+06:45] ALB: i-0a1b890a passed health checks -> InService, receiving traffic
```

Metric	Self-healing (ASG)	Static, manual fix
Time to mark failed instance unhealthy	15 s	15 s (no replacement)
Time until full capacity restored	**105 s**	900 s (assumed)
Requests dropped	3,508 of 3,30,854	19,006 of 3,30,712
Requests served	**98.9%**	94.3%

Discussion: the load balancer stopped routing to the failed instance after 15 s and the group launched a replacement automatically, restoring full capacity in 105 s with no operator action. The remaining losses occur in the short window before the failure is detected and while the single surviving instance is saturated (220 requests/s against a 200 requests/s capacity). Running three instances instead of two would remove the second effect, at higher cost.

17.3 Test 2 - Traffic spike and scale-in

Traffic starts at 200 requests/s, ramps to 720 requests/s between minutes 5 and 10, holds until minute 25, then falls back to 200 requests/s by minute 30.

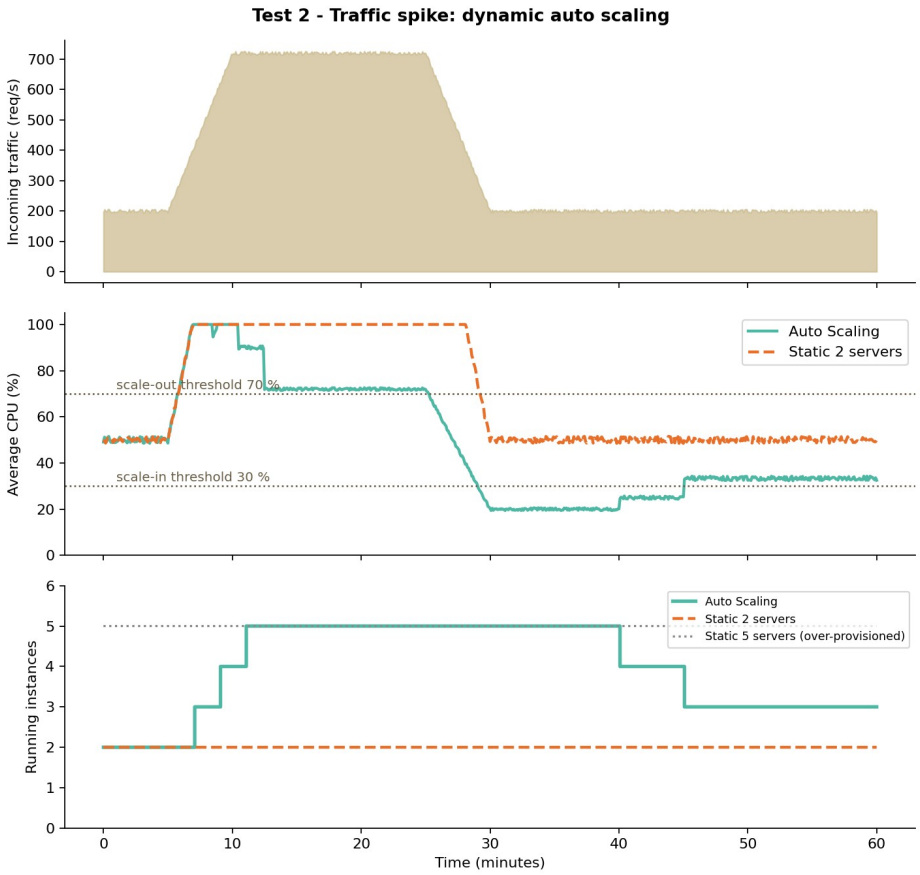


Figure 17.2 - Traffic, average CPU and instance count during the spike

Event log - auto scaling system (simulation output)

```
[T+00:00] ASG started: desired=2, instances=['i-0a1b4b2c', 'i-0a1b6a1b']
[T+07:00] CloudWatch ALARM cpu-high (avg>70% x2 min) -> ASG scale-OUT, launching i-0a1b890a
[T+08:30] ALB: i-0a1b890a passed health checks -> InService, receiving traffic
[T+09:00] CloudWatch ALARM cpu-high (avg>70% x2 min) -> ASG scale-OUT, launching i-0a1ba7f9
[T+10:30] ALB: i-0a1ba7f9 passed health checks -> InService, receiving traffic
[T+11:00] CloudWatch ALARM cpu-high (avg>70% x2 min) -> ASG scale-OUT, launching i-0a1bc6e8
[T+12:30] ALB: i-0a1bc6e8 passed health checks -> InService, receiving traffic
[T+40:00] CloudWatch ALARM cpu-low (avg<30% x2x5 min) -> ASG scale-IN, terminating i-0a1bc6e8
[T+45:00] CloudWatch ALARM cpu-low (avg<30% x2x5 min) -> ASG scale-IN, terminating i-0a1ba7f9
```

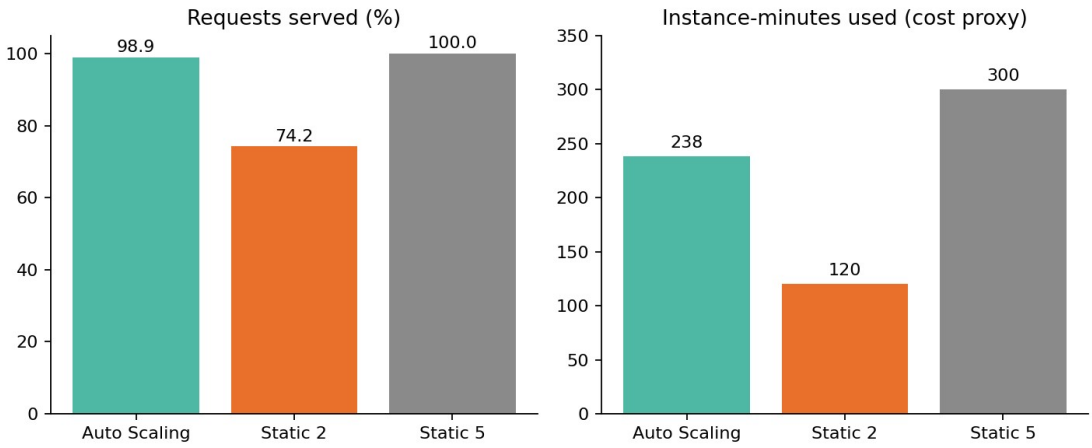


Figure 17.3 - Requests served and instance-minutes for three deployment strategies

Metric	Auto Scaling (2-5)	Static 2 servers	Static 5 servers
Requests served	**98.9%**	74.2%	100%
Requests dropped	14,695	3,47,244	0
Peak instances	5	2	5
Instance-minutes (cost proxy)	**238.2**	120.2	300.4

Discussion: the CPU alarm fired at minute 7, two minutes into the ramp, and further instances were launched at minutes 7, 9 and 11, reaching the maximum of five. The auto-scaled system served 98.9% of requests versus 74.2% for two fixed servers. A static five-server fleet served everything but used 300.4 instance-minutes, whereas Auto Scaling used 238.2 - 20.7% fewer. After the spike, the group scaled in from five to three instances, and stopped there because CPU with three instances (about 33%) is above the 30% scale-in threshold.

Two limitations are visible in the results. First, scaling is **reactive**: capacity arrives only after CPU rises and a new instance boots, so some requests are lost during the ramp - predictive or scheduled scaling (Section 20) addresses this. Second, at the maximum of five instances, CPU remained slightly above the 70% threshold, which shows that the maximum size must be chosen with expected peak load in mind.

17.4 Results from the live AWS deployment

After deploying the stack, record the measurements below and insert the screenshots. The simulation figures above are the expected behaviour; real values depend on instance type, AMI boot time and account limits.

Measurement (live AWS)	Expected (prototype)	Observed
Failure detected (unhealthy) after	about 15-30 s	[fill in]
Replacement InService after	about 105 s	[fill in]
Scale-out alarm fired at average CPU	> 70% for 2 min	[fill in]
Peak instance count during load test	up to 5	[fill in]

Measurement (live AWS)	Expected (prototype)	Observed
Requests failed during Test 1 (load_test.py)	few / none	[fill in]

[INSERT SCREENSHOT] Figure 17.4 - CloudFormation stack

Stack self-healing-demo showing CREATE_COMPLETE, and the Outputs tab with the load balancer URL

[INSERT SCREENSHOT] Figure 17.5 - Application in browser

Browser at the ALB URL; refresh to show different instance IDs / Availability Zones (load balancing)

[INSERT SCREENSHOT] Figure 17.6 - Target group health

Registered targets in both AZs showing 'healthy', then one 'unhealthy' during Test 1

[INSERT SCREENSHOT] Figure 17.7 - Auto Scaling activity history

Activity tab showing the termination and the automatic launch of the replacement instance

[INSERT SCREENSHOT] Figure 17.8 - CloudWatch alarm and scale-out

cpu-high alarm in ALARM state, and the group's instance count rising during the load test

18. Advantages and Limitations

Advantages

- **High availability** - failed instances are detected and replaced automatically
- **Better performance** - extra capacity is added during peak traffic
- **Cost savings** - capacity is reduced when demand falls; in the prototype 20.7% fewer instance-minutes than a statically over-provisioned fleet
- **Reduced manual effort** - no continuous server monitoring by administrators
- **Scalability and flexibility** - the same design grows from a few instances to many
- **Fault tolerance** - multi-AZ placement protects against zone-level failures

Limitations

- **Configuration complexity** - thresholds, cooldowns and health-check settings need careful tuning
- **Scaling delay** - new instances need time to boot and pass health checks, so sudden spikes can briefly affect users
- **Stateless design needed** - session data and files must live in shared storage or a database, not on one instance
- **Cost risks** - wrong scaling rules can launch too many instances
- **AWS dependency** - the design relies on AWS-specific services
- **Limited coverage** - health checks detect instance and application failures but not every logical or data-level problem
- **Prototype limits** - the simulation results rest on assumed capacity and boot-time values

---

19. Applications

- E-commerce websites with sale-driven traffic spikes
- University and exam-result portals with predictable peaks
- Online learning and video platforms
- Banking and payment front-ends that must stay available
- Start-up web and mobile back-ends that need to grow without re-architecture
- Any stateless web service that must tolerate instance failure

---

20. Future Scope

- **Predictive and scheduled scaling** - add capacity before known peaks to remove the reactive delay seen in Test 2
- **Target-tracking policies** - track a target such as 50% CPU or requests-per-target instead of fixed alarms
- **Containers and Kubernetes** - move to Amazon ECS or EKS for faster scaling and better utilisation
- **Multi-region disaster recovery** - Route 53 failover to survive a regional outage

- **Advanced monitoring and security** - centralized logging, dashboards, AWS WAF, and automated remediation using Lambda
- **Private subnets with NAT** and HTTPS (ACM certificate) for a production-grade deployment

21. Conclusion

This project demonstrates how AWS services work together to build an infrastructure that monitors itself, repairs itself, and adjusts to demand without manual intervention. EC2 provides the compute capacity, the Application Load Balancer distributes traffic and checks health, the Auto Scaling Group replaces failed instances and manages capacity, and CloudWatch supplies the metrics and alarms that drive these decisions.

In the prototype, the self-healing mechanism restored full capacity 105 seconds after an instance failure, and auto scaling kept 98.9% of requests served during a 3.6x traffic spike while using 20.7% fewer instance-minutes than a fixed fleet sized for the peak. Careful configuration and a stateless application design are required, but the benefits outweigh the challenges. With predictive scaling, Infrastructure as Code (already included here), and multi-region deployment, the solution can grow into a production-grade, fully automated cloud platform.

22. References

[1] Amazon Web Services, “Amazon EC2 Auto Scaling User Guide.” https://docs.aws.amazon.com/autoscaling

[2] Amazon Web Services, “Elastic Load Balancing User Guide.” https://docs.aws.amazon.com/elasticloadbalancing

[3] Amazon Web Services, “Amazon CloudWatch User Guide.” https://docs.aws.amazon.com/cloudwatch

[4] Amazon Web Services, “Amazon EC2 User Guide.” https://docs.aws.amazon.com/ec2

[5] Amazon Web Services, “AWS CloudFormation User Guide.” https://docs.aws.amazon.com/cloudformation

[6] Amazon Web Services, “AWS Well-Architected Framework - Reliability Pillar.” https://docs.aws.amazon.com/wellarchitected

[7] M. Armbrust et al., “A View of Cloud Computing,” Communications of the ACM, vol. 53, no. 4, pp. 50-58, 2010.

[8] P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” NIST Special Publication 800-145, 2011.

[9] Netflix, “Chaos Monkey.” https://netflix.github.io/chaosmonkey

Appendix A - Complete CloudFormation Template

cloudformation/self-healing-asg.yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Self-Healing Infrastructure with Auto Scaling (VPC, ALB, ASG, CloudWatch, SNS)

Parameters:
  InstanceType:
    Type: String
    Default: t2.micro
  LatestAmiId:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2
  AlertEmail:
    Type: String
    Description: Email address that receives scaling/failure notifications

Resources:
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 10.0.0.0/16
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags: [{Key: Name, Value: self-healing-vpc}]

  IGW:
    Type: AWS::EC2::InternetGateway
  IGWAttach:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties: {VpcId: !Ref VPC, InternetGatewayId: !Ref IGW}

  PublicSubnetA:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: 10.0.1.0/24
      AvailabilityZone: !Select [0, !GetAZs '']
      MapPublicIpOnLaunch: true
      Tags: [{Key: Name, Value: public-subnet-a}]
  PublicSubnetB:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: 10.0.2.0/24
      AvailabilityZone: !Select [1, !GetAZs '']
      MapPublicIpOnLaunch: true
      Tags: [{Key: Name, Value: public-subnet-b}]

  RouteTable:
    Type: AWS::EC2::RouteTable
    Properties: {VpcId: !Ref VPC}
  DefaultRoute:
    Type: AWS::EC2::Route
    DependsOn: IGWAttach
    Properties:
      RouteTableId: !Ref RouteTable
```

```
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref IGW
RTAssocA:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties: {SubnetId: !Ref PublicSubnetA, RouteTableId: !Ref RouteTable}
RTAssocB:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties: {SubnetId: !Ref PublicSubnetB, RouteTableId: !Ref RouteTable}

AlbSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Allow HTTP from the internet to the ALB
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - {IpProtocol: tcp, FromPort: 80, ToPort: 80, CidrIp: 0.0.0.0/0}
InstanceSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Allow HTTP only from the ALB
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - {IpProtocol: tcp, FromPort: 80, ToPort: 80, SourceSecurityGroupId: !Ref AlbSecurityGroup}

LoadBalancer:
  Type: AWS::ElasticLoadBalancingV2::LoadBalancer
  Properties:
    Type: application
    Scheme: internet-facing
    SecurityGroups: [!Ref AlbSecurityGroup]
    Subnets: [!Ref PublicSubnetA, !Ref PublicSubnetB]
TargetGroup:
  Type: AWS::ElasticLoadBalancingV2::TargetGroup
  Properties:
    VpcId: !Ref VPC
    Port: 80
    Protocol: HTTP
    HealthCheckPath: /health.html
    HealthCheckIntervalSeconds: 15
    HealthCheckTimeoutSeconds: 5
    HealthyThresholdCount: 2
    UnhealthyThresholdCount: 2
    TargetGroupAttributes:
      - {Key: deregistration_delay.timeout_seconds, Value: '30'}
Listener:
  Type: AWS::ElasticLoadBalancingV2::Listener
  Properties:
    LoadBalancerArn: !Ref LoadBalancer
    Port: 80
    Protocol: HTTP
    DefaultActions:
      - {Type: forward, TargetGroupArn: !Ref TargetGroup}

LaunchTemplate:
  Type: AWS::EC2::LaunchTemplate
  Properties:
    LaunchTemplateName: self-healing-lt
    LaunchTemplateData:
      ImageId: !Ref LatestAmiId
      InstanceType: !Ref InstanceType
```

```
        SecurityGroupIds: [!Ref InstanceSecurityGroup]
        UserData:
          Fn::Base64: |
            #!/bin/bash
            yum update -y
            yum install -y httpd stress
            systemctl enable --now httpd
            TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-
token-ttl-seconds: 21600")
            IID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-
data/instance-id)
            AZ=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" http://169.254.169.254/latest/meta-
data/placement/availability-zone)
            echo "<h1>Self-Healing Infrastructure</h1><h2>Instance: $IID</h2><h3>AZ: $AZ</h3>" >
/var/www/html/index.html
            echo "OK" > /var/www/html/health.html

  AutoScalingGroup:
    Type: AWS::AutoScaling::AutoScalingGroup
    Properties:
      AutoScalingGroupName: self-healing-asg
      MinSize: '2'
      MaxSize: '5'
      DesiredCapacity: '2'
      HealthCheckType: ELB
      HealthCheckGracePeriod: 120
      VPCZoneIdentifier: [!Ref PublicSubnetA, !Ref PublicSubnetB]
      TargetGroupARNs: [!Ref TargetGroup]
      LaunchTemplate:
        LaunchTemplateId: !Ref LaunchTemplate
        Version: !GetAtt LaunchTemplate.LatestVersionNumber
      NotificationConfigurations:
        - TopicARN: !Ref AlertTopic
          NotificationTypes:
            - autoscaling:EC2_INSTANCE_LAUNCH
            - autoscaling:EC2_INSTANCE_TERMINATE
            - autoscaling:EC2_INSTANCE_LAUNCH_ERROR

  ScaleOutPolicy:
    Type: AWS::AutoScaling::ScalingPolicy
    Properties:
      AutoScalingGroupName: !Ref AutoScalingGroup
      PolicyType: SimpleScaling
      AdjustmentType: ChangeInCapacity
      ScalingAdjustment: 1
      Cooldown: '120'
  ScaleInPolicy:
    Type: AWS::AutoScaling::ScalingPolicy
    Properties:
      AutoScalingGroupName: !Ref AutoScalingGroup
      PolicyType: SimpleScaling
      AdjustmentType: ChangeInCapacity
      ScalingAdjustment: -1
      Cooldown: '300'

  HighCpuAlarm:
    Type: AWS::CloudWatch::Alarm
    Properties:
      AlarmName: self-healing-cpu-high
      Namespace: AWS/EC2
```

```
      MetricName: CPUUtilization
      Dimensions: [{Name: AutoScalingGroupName, Value: !Ref AutoScalingGroup}]
      Statistic: Average
      Period: 60
      EvaluationPeriods: 2
      Threshold: 70
      ComparisonOperator: GreaterThanThreshold
      AlarmActions: [!Ref ScaleOutPolicy, !Ref AlertTopic]
  LowCpuAlarm:
    Type: AWS::CloudWatch::Alarm
    Properties:
      AlarmName: self-healing-cpu-low
      Namespace: AWS/EC2
      MetricName: CPUUtilization
      Dimensions: [{Name: AutoScalingGroupName, Value: !Ref AutoScalingGroup}]
      Statistic: Average
      Period: 300
      EvaluationPeriods: 2
      Threshold: 30
      ComparisonOperator: LessThanThreshold
      AlarmActions: [!Ref ScaleInPolicy]

  AlertTopic:
    Type: AWS::SNS::Topic
    Properties:
      Subscription:
        - {Protocol: email, Endpoint: !Ref AlertEmail}

Outputs:
  LoadBalancerDNS:
    Description: Open this URL in a browser
    Value: !Sub http://${LoadBalancer.DNSName}
```

Appendix B - Prototype Simulation Source Code

simulation/simulate.py

```
"""
Local prototype simulation of the Self-Healing Infrastructure with Auto Scaling.

Models an Application Load Balancer + Auto Scaling Group + CloudWatch alarms using the SAME
parameters as cloudformation/self-healing-asg.yaml:
  min=2, desired=2, max=5 | ELB health check every 15 s, unhealthy threshold 2, healthy threshold 2
  scale-out: avg CPU > 70 % for 2 x 60 s  -> +1 instance, cooldown 120 s
  scale-in : avg CPU < 30 % for 2 x 300 s -> -1 instance, cooldown 300 s
  instance boot time (launch + user-data) ~ 75 s

Two experiments:
  TEST 1 - instance failure (steady traffic, one instance crashes)
  TEST 2 - traffic spike   (traffic ramps up, holds, then falls)
Each is compared with a STATIC deployment (2 fixed servers, no self-healing / no scaling).

NOTE: this is a model, not a measurement of real AWS. Capacity per instance is an assumption.
Run:  python simulate.py        (writes CSV, event logs and PNG charts to ./output)
"""
import os, csv, random
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

random.seed(7)
OUT = os.path.join(os.path.dirname(os.path.abspath(__file__)), "output")
os.makedirs(OUT, exist_ok=True)

STEP = 5                      # seconds per simulation tick
CAPACITY = 200.0              # requests/s one instance can serve at 100 % CPU (assumption)
BOOT = 75                     # seconds from launch to web server up
HC_INTERVAL, UNHEALTHY_N, HEALTHY_N = 15, 2, 2
MIN_N, MAX_N = 2, 5
OUT_CPU, IN_CPU = 70, 30
OUT_COOL, IN_COOL = 120, 300
MANUAL_FIX = 900              # assumed time (s) for an admin to notice and fix a failed static server


class Inst:
    _n = 0
    def __init__(self, t, booted=False):
        Inst._n += 1
        self.id = f"i-{0x0a1b2c3d + Inst._n * 7919:08x}"
        self.t_launch = t
        self.up_at = t if booted else t + BOOT      # web server ready
        self.alive = True
        self.in_tg = booted                           # registered & healthy in target group
        self.hc_ok = HEALTHY_N if booted else 0       # consecutive successes
        self.hc_fail = 0
        self.az = "ap-south-1a" if Inst._n % 2 else "ap-south-1b"
    def serving(self, t):
        return self.alive and t >= self.up_at


def run(profile, duration, self_heal=True, autoscale=True, fail_at=None, static_n=2, name=""):
```

```
    Inst._n = 0
    insts = [Inst(0, True) for _ in range(static_n)]
    events, rows = [], []
    def ev(t, msg): events.append((t, msg))
    ev(0, f"ASG started: desired={static_n}, instances={[i.id for i in insts]}")
    cpu_hist, low_hist = [], []
    last_out = last_in = -10**9
    desired = static_n
    failed = None
    t_fail_fixed = None
    for t in range(0, duration + 1, STEP):
        # ---- failure injection
        if fail_at is not None and t == fail_at and failed is None:
            failed = insts[0]; failed.alive = False
            ev(t, f"FAILURE injected: {failed.id} ({failed.az}) crashed")
            if not self_heal: t_fail_fixed = t + MANUAL_FIX
        if (not self_heal) and failed is not None and t_fail_fixed and t >= t_fail_fixed and not
failed.alive:
            failed.alive = True; failed.hc_ok = HEALTHY_N
            ev(t, f"MANUAL fix: admin restarted {failed.id}")
        # ---- ELB health checks (every HC_INTERVAL s)
        if t % HC_INTERVAL == 0:
            for i in list(insts):
                ok = i.serving(t)
                if ok: i.hc_ok += 1; i.hc_fail = 0
                else:  i.hc_fail += 1; i.hc_ok = 0
                if not i.in_tg and i.hc_ok >= HEALTHY_N:
                    i.in_tg = True; ev(t, f"ALB: {i.id} passed health checks -> InService, receiving
traffic")
                if i.in_tg and i.hc_fail >= UNHEALTHY_N:
                    i.in_tg = False; ev(t, f"ALB: {i.id} FAILED {UNHEALTHY_N} health checks -> marked
Unhealthy, traffic stopped")
                    if self_heal:
                        insts.remove(i)
                        ev(t, f"ASG: terminating unhealthy {i.id}")
                        n = Inst(t); insts.append(n)
                        ev(t, f"ASG: launching replacement {n.id} ({n.az}) from launch template")
        # ---- traffic distribution
        load = max(0.0, profile(t) + random.uniform(-6, 6))
        targets = [i for i in insts if i.in_tg]
        dead_targets = [i for i in targets if not i.serving(t)]       # ALB hasn't noticed yet
        good = [i for i in targets if i.serving(t)]
        if targets:
            share = load / len(targets)
            lost = share * len(dead_targets)
            per = share
            served_cap = sum(min(per, CAPACITY) for _ in good)
            served = served_cap
            dropped = lost + sum(max(0, per - CAPACITY) for _ in good)
            cpu = min(100.0, per / CAPACITY * 100) if good else 0.0
        else:
            served, dropped, cpu = 0.0, load, 0.0
        avail = 100.0 * served / load if load > 0 else 100.0
        rows.append(dict(t=t, traffic=round(load, 1), instances=len([i for i in insts if i.alive]),
                         in_service=len(good), cpu=round(cpu, 1), served=round(served, 1),
                         dropped=round(dropped, 1), availability=round(avail, 1)))
        # ---- CloudWatch alarms (evaluated once a minute)
        if autoscale and t % 60 == 0 and t > 0:
            cpu_hist.append(cpu); low_hist.append(cpu)
            if len(cpu_hist) >= 2 and all(c > OUT_CPU for c in cpu_hist[-2:]):
```

```
                if len([i for i in insts]) < MAX_N and t - last_out >= OUT_COOL:
                    n = Inst(t); insts.append(n); last_out = t
                    ev(t, f"CloudWatch ALARM cpu-high (avg>{OUT_CPU}% x2 min) -> ASG scale-OUT,
launching {n.id}")
            if t % 300 == 0 and len(low_hist) >= 10:
                avg_a = sum(low_hist[-10:-5]) / 5; avg_b = sum(low_hist[-5:]) / 5
                if avg_a < IN_CPU and avg_b < IN_CPU and len(insts) > MIN_N and t - last_in >= IN_COOL:
                    victim = insts[-1]; insts.remove(victim); last_in = t
                    ev(t, f"CloudWatch ALARM cpu-low (avg<{IN_CPU}% x2x5 min) -> ASG scale-IN,
terminating {victim.id}")
    return rows, sorted(events, key=lambda e: e[0])


def fmt(t): return f"{t//60:02d}:{t%60:02d}"
def save(name, rows, events):
    with open(f"{OUT}/{name}.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=rows[0].keys()); w.writeheader(); w.writerows(rows)
    with open(f"{OUT}/{name}_events.log", "w") as f:
        for t, m in events: f.write(f"[T+{fmt(t)}] {m}\n")

def summary(rows):
    tot = sum(r["traffic"] for r in rows); drop = sum(r["dropped"] for r in rows)
    return dict(total_requests=tot * STEP, dropped=drop * STEP, availability=100 * (1 - drop / tot),
                peak_cpu=max(r["cpu"] for r in rows),
                instance_minutes=sum(r["instances"] for r in rows) * STEP / 60,
                max_instances=max(r["instances"] for r in rows),
                min_avail_tick=min(r["availability"] for r in rows))

# ---------------- TEST 1: instance failure ----------------
steady = lambda t: 220.0
D1, FAIL = 1500, 300
a1, e1 = run(steady, D1, True, True, FAIL, name="t1")
b1, eb1 = run(steady, D1, False, False, FAIL, name="t1_static")
save("test1_self_healing", a1, e1); save("test1_static_baseline", b1, eb1)

# ---------------- TEST 2: traffic spike ----------------
def spike(t):
    if t < 300: return 200
    if t < 600: return 200 + (t - 300) / 300 * 520      # ramp to 720 req/s
    if t < 1500: return 720
    if t < 1800: return 720 - (t - 1500) / 300 * 520    # fall back
    return 200
D2 = 3600
a2, e2 = run(spike, D2, True, True, None, name="t2")
b2, eb2 = run(spike, D2, True, False, None, name="t2_static2")
p2, ep2 = run(spike, D2, True, False, None, static_n=5, name="t2_static5")
save("test2_autoscaling", a2, e2); save("test2_static_2_servers", b2, eb2);
save("test2_static_5_servers", p2, ep2)

S = {k: summary(v) for k, v in dict(t1=a1, t1s=b1, t2=a2, t2s=b2, t2p=p2).items()}
with open(f"{OUT}/summary.txt", "w") as f:
    for k, v in S.items():
        f.write(f"{k}: " + ", ".join(f"{a}={b:.1f}" if isinstance(b, float) else f"{a}={b}" for a, b in
v.items()) + "\n")
print(open(f"{OUT}/summary.txt").read())

# ---------------- charts ----------------
C = dict(tan="#c9b98a", teal="#4fb8a5", orange="#e8702a", brown="#695d46", grey="#8a8a8a")
plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False})
```

```
def chart_failure():
    fig, ax = plt.subplots(3, 1, figsize=(9, 8), sharex=True)
    x = [r["t"] / 60 for r in a1]
    ax[0].plot(x, [r["instances"] for r in a1], color=C["teal"], lw=2.2, drawstyle="steps-post",
label="Self-healing (ASG)")
    ax[0].plot(x, [r["instances"] for r in b1], color=C["orange"], lw=2.2, ls="--", drawstyle="steps-
post", label="Static (no self-healing)")
    ax[0].set_ylabel("Running instances"); ax[0].set_ylim(0, 3.5); ax[0].legend(loc="lower right")
    ax[1].plot(x, [r["in_service"] for r in a1], color=C["teal"], lw=2.2, drawstyle="steps-post")
    ax[1].plot(x, [r["in_service"] for r in b1], color=C["orange"], lw=2.2, ls="--", drawstyle="steps-
post")
    ax[1].set_ylabel("InService in target group"); ax[1].set_ylim(0, 3.5)
    ax[2].plot(x, [r["availability"] for r in a1], color=C["teal"], lw=2.2)
    ax[2].plot(x, [r["availability"] for r in b1], color=C["orange"], lw=2.2, ls="--")
    ax[2].set_ylabel("Requests served (%)"); ax[2].set_xlabel("Time (minutes)"); ax[2].set_ylim(0, 105)
    for a in ax: a.axvline(FAIL / 60, color=C["brown"], ls=":", lw=1.3)
    ax[0].text(FAIL / 60 + 0.2, 3.1, "instance crash", color=C["brown"])
    fig.suptitle("Test 1 - Instance failure: self-healing vs static deployment", fontweight="bold")
    fig.tight_layout(); fig.savefig(f"{OUT}/chart_test1_failure.png", dpi=160); plt.close(fig)

def chart_spike():
    fig, ax = plt.subplots(3, 1, figsize=(9, 8.6), sharex=True)
    x = [r["t"] / 60 for r in a2]
    ax[0].fill_between(x, [r["traffic"] for r in a2], color=C["tan"], alpha=.7)
    ax[0].set_ylabel("Incoming traffic (req/s)")
    ax[1].plot(x, [r["cpu"] for r in a2], color=C["teal"], lw=2, label="Auto Scaling")
    ax[1].plot(x, [r["cpu"] for r in b2], color=C["orange"], lw=2, ls="--", label="Static 2 servers")
    ax[1].axhline(70, color=C["brown"], ls=":", lw=1.2); ax[1].text(1, 72, "scale-out threshold 70 %",
color=C["brown"], fontsize=9)
    ax[1].axhline(30, color=C["brown"], ls=":", lw=1.2); ax[1].text(1, 32, "scale-in threshold 30 %",
color=C["brown"], fontsize=9)
    ax[1].set_ylabel("Average CPU (%)"); ax[1].legend(loc="upper right"); ax[1].set_ylim(0, 105)
    ax[2].plot(x, [r["instances"] for r in a2], color=C["teal"], lw=2.4, drawstyle="steps-post",
label="Auto Scaling")
    ax[2].plot(x, [r["instances"] for r in b2], color=C["orange"], lw=2, ls="--", drawstyle="steps-
post", label="Static 2 servers")
    ax[2].plot(x, [r["instances"] for r in p2], color=C["grey"], lw=1.5, ls=":", drawstyle="steps-
post", label="Static 5 servers (over-provisioned)")
    ax[2].set_ylabel("Running instances"); ax[2].set_xlabel("Time (minutes)"); ax[2].set_ylim(0, 6);
ax[2].legend(loc="upper right", fontsize=8)
    fig.suptitle("Test 2 - Traffic spike: dynamic auto scaling", fontweight="bold")
    fig.tight_layout(); fig.savefig(f"{OUT}/chart_test2_scaling.png", dpi=160); plt.close(fig)

def chart_bars():
    fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
    labels = ["Auto Scaling", "Static 2", "Static 5"]
    av = [S["t2"]["availability"], S["t2s"]["availability"], S["t2p"]["availability"]]
    im = [S["t2"]["instance_minutes"], S["t2s"]["instance_minutes"], S["t2p"]["instance_minutes"]]
    b = ax[0].bar(labels, av, color=[C["teal"], C["orange"], C["grey"]]); ax[0].set_ylim(0, 105);
ax[0].set_title("Requests served (%)")
    for r, v in zip(b, av): ax[0].text(r.get_x() + r.get_width() / 2, v + 1.5, f"{v:.1f}", ha="center")
    b = ax[1].bar(labels, im, color=[C["teal"], C["orange"], C["grey"]]); ax[1].set_title("Instance-
minutes used (cost proxy)"); ax[1].set_ylim(0, 350)
    for r, v in zip(b, im): ax[1].text(r.get_x() + r.get_width() / 2, v + 8, f"{v:.0f}", ha="center")
    fig.tight_layout(); fig.savefig(f"{OUT}/chart_comparison.png", dpi=160); plt.close(fig)

chart_failure(); chart_spike(); chart_bars()
print("files:", sorted(os.listdir(OUT)))
```